



Big Data Practical Project (Stock Price Analysis)

By: Shahd Ahmed Tageldin

GitHub Repo: <https://github.com/shahdAhmed101/Big-Data-Stock-Price-Analysis>

▼ Dataset

The used data is the "Stock Prices" data set from Kaggle, representing **historical stock market data** for current S&P 500 companies, from 2014-2017. Each record represents a single day of trading, and includes the ticker name, volume, high, low, open and close prices.

<https://www.kaggle.com/datasets/mysarahmadbhat/stock-prices>

```
C: > Users > shahd > AppData > Local > Temp > 1b0b7430-e460-4927-8bb9-b8797ee5cfab_archive (2).zip.fab >
1   symbol,date,open,high,low,close,volume
2   AAL,2014-01-02,25.07,25.82,25.06,25.36,8998943
3   AAPL,2014-01-02,79.3828,79.5756,78.8601,79.0185,58791957
4   AAP,2014-01-02,110.36,111.88,109.29,109.74,542711
5   ABBV,2014-01-02,52.12,52.33,51.52,51.98,4569061
6   ABC,2014-01-02,70.11,70.23,69.48,69.89,1148391
7   ABT,2014-01-02,38.09,38.4,38,38.23,4967472
8   ACN,2014-01-02,81.5,81.92,81.09,81.13,2405384
9   ADBE,2014-01-02,59.06,59.53,58.94,59.29,2746370
10  ADI,2014-01-02,49.52,49.75,49.04,49.28,2799092
```

▼ Real-time Simulation

The goal is to split the entire dataset into batches (according to the "date" column) so we can use it to simulate new data arriving every day.

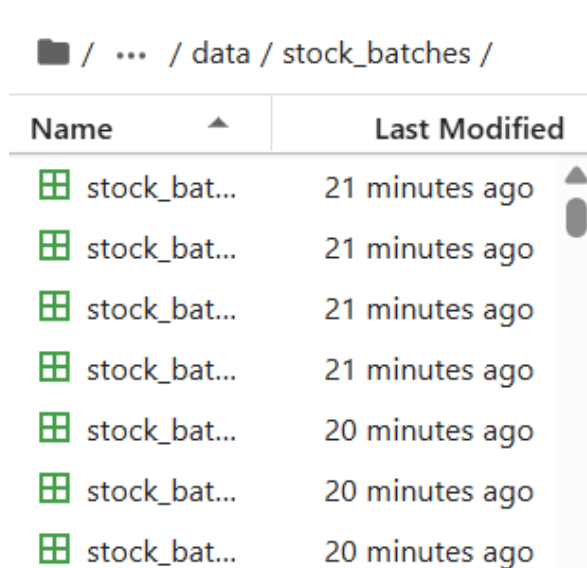
This is done by:

- 1) Taking the static csv (containing all the data at once)
- 2) Breaking the data into smaller batches, **similar to how a real system would add a new file by the end of each day.**

```
Batch 997/1007: 2017-12-14 → 505 records | File: stock_batch_2017-12-14.csv
Batch 998/1007: 2017-12-15 → 505 records | File: stock_batch_2017-12-15.csv
Batch 999/1007: 2017-12-18 → 505 records | File: stock_batch_2017-12-18.csv
Batch 1000/1007: 2017-12-19 → 505 records | File: stock_batch_2017-12-19.csv
Batch 1001/1007: 2017-12-20 → 505 records | File: stock_batch_2017-12-20.csv
Batch 1002/1007: 2017-12-21 → 505 records | File: stock_batch_2017-12-21.csv
Batch 1003/1007: 2017-12-22 → 505 records | File: stock_batch_2017-12-22.csv
Batch 1004/1007: 2017-12-26 → 505 records | File: stock_batch_2017-12-26.csv
Batch 1005/1007: 2017-12-27 → 505 records | File: stock_batch_2017-12-27.csv
Batch 1006/1007: 2017-12-28 → 505 records | File: stock_batch_2017-12-28.csv
Batch 1007/1007: 2017-12-29 → 505 records | File: stock_batch_2017-12-29.csv
```

Done! All batches saved in '/home/jovyan/work/data/stock_batches'

The result of running the real world simulation file



/ ... / data / stock_batches /	
Name	Last Modified
stock_bat...	21 minutes ago
stock_bat...	21 minutes ago
stock_bat...	21 minutes ago
stock_bat...	21 minutes ago
stock_bat...	20 minutes ago
stock_bat...	20 minutes ago
stock_bat...	20 minutes ago

Saved data batches in the created folder

	symbol	date	open	high	low	close	volume	processing_ts
1	AAL	2014-01-02	25.07	25.82	25.06	25.36	8998943	2026-05-06 12:51:10
2	AAPL	2014-01-02	79.3828	79.5756	78.8601	79.0185	58791957	2026-05-06 12:51:10
3	AAP	2014-01-02	110.36	111.88	109.29	109.74	542711	2026-05-06 12:51:10
4	ABBV	2014-01-02	52.12	52.33	51.52	51.98	4569061	2026-05-06 12:51:10
5	ABC	2014-01-02	70.11	70.23	69.48	69.89	1148391	2026-05-06 12:51:10
6	ABT	2014-01-02	38.09	38.4	38.0	38.23	4967472	2026-05-06 12:51:10
7	ACN	2014-01-02	81.5	81.92	81.09	81.13	2405384	2026-05-06 12:51:10
8	ADBE	2014-01-02	59.06	59.53	58.94	59.29	2746370	2026-05-06 12:51:10
9	ADI	2014-01-02	49.52	49.75	49.04	49.28	2799092	2026-05-06 12:51:10
10	ADM	2014-01-02	43.22	43.29	42.79	42.99	2753765	2026-05-06 12:51:10
11	ADP	2014-01-02	80.17	80.45	79.38	79.86	1965869	2026-05-06 12:51:10
12	ADSK	2014-01-02	49.33	49.74	48.88	49.25	2488043	2026-05-06 12:51:10
13	ADS	2014-01-02	262.44	262.68	258.78	262.34	547808	2026-05-06 12:51:10
14	AEE	2014-01-02	36.05	36.11	35.47	35.53	1394264	2026-05-06 12:51:10
15	AEP	2014-01-02	46.98	47.06	46.04	46.13	2752002	2026-05-06 12:51:10
16	AES	2014-01-02	14.51	14.56	14.19	14.27	3987216	2026-05-06 12:51:10
17	AET	2014-01-02	68.21	68.6	67.52	67.56	1650687	2026-05-06 12:51:10
18	AFL	2014-01-02	66.63	66.694	65.81	65.96	1914657	2026-05-06 12:51:10
19	AGN	2014-01-02	167.33	170.71	166.5937	168.05	1268722	2026-05-06 12:51:10
20	AIG	2014-01-02	50.81	51.3	50.47	50.71	9198092	2026-05-06 12:51:10
21	AIV	2014-01-02	25.9	26.14	25.66	25.91	844901	2026-05-06 12:51:10
22	AIZ	2014-01-02	66.04	66.19	65.15	65.32	375524	2026-05-06 12:51:10

Example batch file (all from the same day: 2014-01-02)

▼ Extraction

The goal of this step is to read the daily batches (created in the simulation step) and writes then into **HDFS** as **parquet files** in the **bronze layer of a data lake**. We submit the spark jobs using **YARN** as the cluster resource manager.

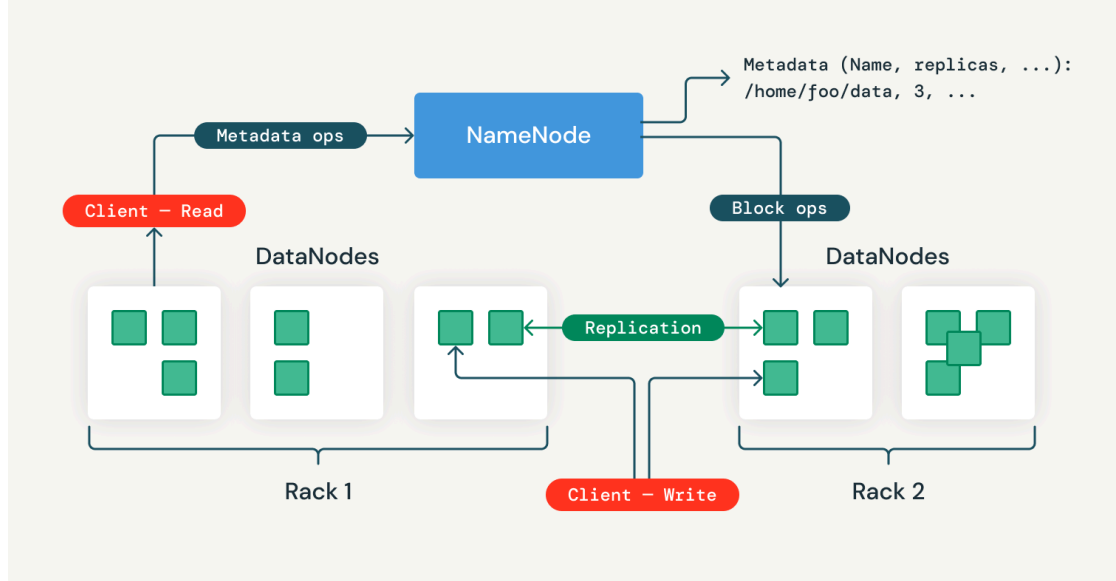
▼ HDFS

HDFS (**H**adoop **D**istributed **F**ile **S**ystem) is a distributed file system that provides access to data across Hadoop clusters. Hadoop stores large files across multiple machines (or in our case, *multiple Docker containers*, simulating separate machines on only 1).

HDFS had 2 main components:

- Namenode
- Datanode

HDFS Architecture



In our "docker-compose" file:

```
hadoop-namenode (172.30.1.21)
hadoop-datanode1 (172.30.1.22)
hadoop-datanode2 (172.30.1.23)
```

This means that HDFS distributes the data blocks across two datanodes (datanode1 & datanode2)

HDFS allows our PySpark scripts to read/write data without worrying about the physical disk the data was stored on.

▼ Parquet Files

Parquet format is a **columnar** storage format. Instead of storing rows one after another (like CSV), it stores data by **columns**. The aim is to support **fast and efficient** data processing and retrieval of complex data.

Impact of Row / Column Storage on Analytics

Example query: what is the average product price?

Row-based storage

Product	Price	Availability
Apples	\$5	High
Oranges	\$3	High
Pears	\$11	Low

Data is scanned sequentially per row.
Calculating the average requires a full table scan

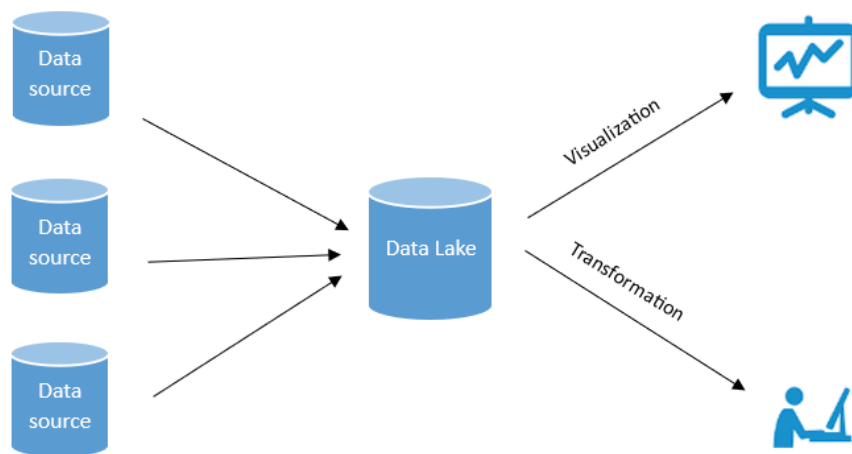
Columnar storage

Products	Price	Availability
Apples	\$5	High
Oranges	\$3	High
Pears	\$11	Low

Calculating the average requires only one column to be scanned

▼ Data Lake

A data lake is a centralized repository that stores vast amounts of raw, unstructured, semi-structured, and structured data **in its native format** (does not require a specific structure), unlike a data warehouse which stores cleaned and processed data **in a certain format**.

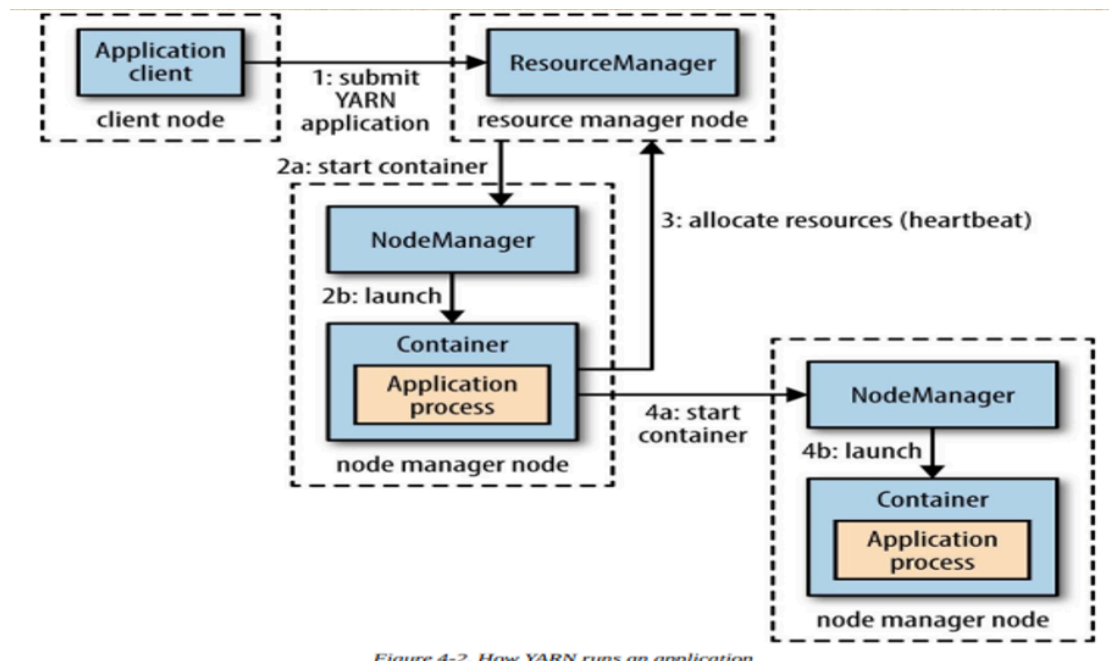


note: HDFS is an example of data lakes.

The medallion architecture organises data into layers:

- **Bronze:** raw data, exactly as it arrived (CSV, JSON). Here, it is **stored as Parquet** for efficiency, but no cleaning/aggregation.
- **Silver:** cleaned and enriched data.
- **Gold:** business-ready tables, usually a star schema (fact & dimension tables) ready for Snowflake.

▼ YARN



YARN (Y**e**t **A**nother **R**esource **N**egotiator) is the cluster manager in Hadoop. It decides which machine (NodeManager) runs the task. Meaning it acts as the **cluster operating system**.

YARN **allocates the necessary resources** and **distributes the workload across different servers(nodes)** in the cluster, optimizing the performance.

We start by building the Spark session:

```
spark = SparkSession.builder \
    .appName('StockBatchIngestion') \
    .master('yarn') \      #means spark will ask YARN for
    resources (and NOT run locally)
    .config("spark.hadoop.fs.defaultFS", "hdfs://hadoop-
    namenode:9000") \
    .config("spark.hadoop.yarn.resourcemanager.hostname", "resourcemanager") \
    .config("spark.hadoop.yarn.resourcemanager.address",
    "resourcemanager:8032") \
    .config("spark.hadoop.yarn.resourcemanager.scheduler.address", "resourcemanager:8030") \
```

```
.config("spark.driver.host", "172.30.1.13") \
.config("spark.driver.bindAddress", "0.0.0.0") \
.config("spark.executor.memory", "512m") \
.config("spark.yarn.am.memory", "512m") \
.getOrCreate()
```

A Spark session is a driver process through which we control our Spark application.

Then defining the data (input) path & HDFS output path (where the parquet files will be stored):

```
input_path = "file:///home/jovyan/work/data/stock_batches/"
hdfs_output_path = "hdfs://hadoop-namenode:9000/user/root/datalake/bronze/stock_data/"
```

Inside the scope of "try" (to catch any errors), we read all the CSVs files (representing the batches created by the simulation file):

```
raw_df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \      #spark will guess
    the columns data types
    .csv(input_path)
```

We then count the records (rows):

```
record_count = raw_df.count()
```

The dataframe (created using "spark.read") is then written to HDFS as parquet file format:

```
raw_df.write \
    .mode("append") \
    .format("parquet") \
    .save(hdfs_output_path)
```

The mode is set to append so, when the extraction file is read multiple times, (simulating real-life behavior, as we typically read the daily new data files that arrived by the end of that same day) it will add the new data **without overwriting/deleting any already-existing data**.

Error handling and termination:

```
except Exception as e:
    print(f"Error during Spark processing: {e}")

finally:
    spark.stop()
    print("Spark Session Stopped.")
```

Cell output:

```
Spark Connected Successfully
Reading batch data from: file:///home/jovyan/work/data/stock_batches/
```

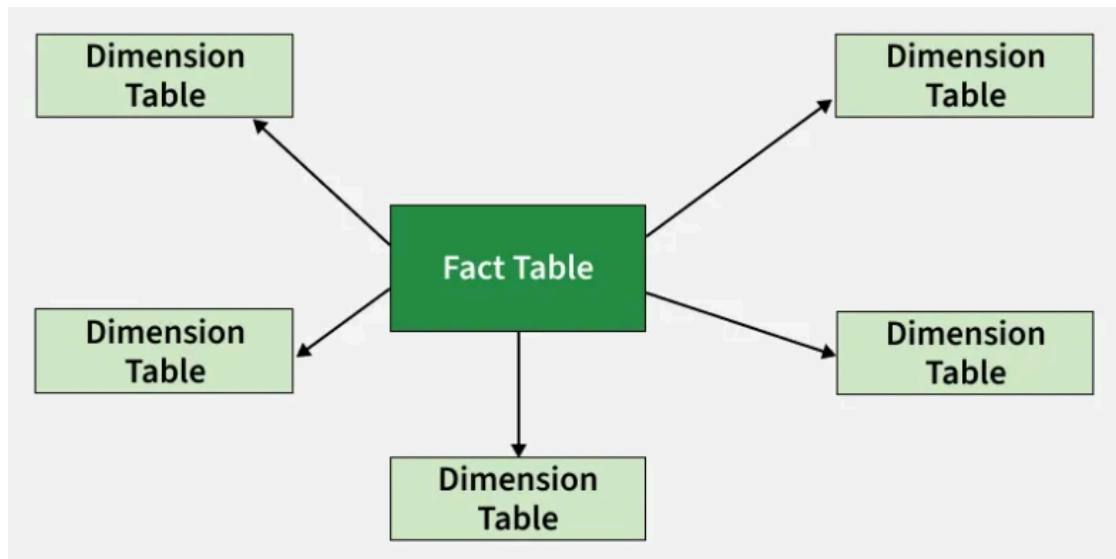
The cell output indicating that we have successfully read the batch data from the previous step.

▼ Transformation

The goal of this step is to clean, format, and process the raw data to match the **star schema**, required by the GOLD layer of the data lake.

▼ Star Schema

A Star Schema organizes data into a **central fact table** linked to **multiple dimension tables**.



A fact table contains **business measurements** (The numbers. e.g. stock prices & volume), while dimension tables describe the fact table, giving context for filtering and grouping.

Dimension tables are typically *denormalized*, which means that fewer JOIN operations are required, making analytical queries more efficient. These dimension tables allow us to slice, dice, filter, and group the fact data for analysis.

Like before we set up the spark session, using the same configurations.

Generate Static Data Dimension:

```

def generate_static_date_dim(spark, start_date="2014-01-01", end_date="2030-12-31"):
    df = spark.sql(f"SELECT CAST('{start_date}' AS DATE) as start, CAST('{end_date}' AS DATE) as end")
    df = df.select(
        F.explode(
            F.sequence(F.to_date("start"), F.to_date("end"), F.expr("interval 1 day"))
        ).alias("date")
    )

    dim_date = df.select(
        F.date_format("date", "yyyyMMdd").cast("int").alias("date_key"),

```

```

        "date",
        F.year("date").alias("year"),
        F.month("date").alias("month"),
        F.dayofmonth("date").alias("day"),
        F.date_format("date", "EEEE").alias("day_name"),
        F.dayofweek("date").alias("day_of_week"),
        F.weekofyear("date").alias("week_of_year"),
        F.quarter("date").alias("quarter"),
        F.when(F.dayofweek("date").isin(1, 7), True).otherwise(False).alias("is_weekend")
    )
    return dim_date

```

Creates a **dim_date** table that contains **one row for every single calendar day between "start_date" and "end_date"**. It doesn't depend on the data; it's a **static dimension** (also called a date dimension or calendar dimension).

Reading the raw stock data:

```
df_bronze = spark.read.parquet(HDFS_BRONZE_PATH)
```

This simply reads all the parquet files that the extraction script wrote into HDFS.

Fixing the "date" column:

```
df_bronze = df_bronze.withColumn("business_date", F.to_date("date"))
```

Up until this point, the column "date" was treated as string, so "to_date" converts it to a real date type, and stores the result into a new column "business_date".

Creating the calendar dimension table "dim-date":

```

try:
    spark.read.parquet(f"{GOLD_BASE_PATH}dim_date")

```

```

    print("dim_date already exists. Skipping generation.")
except Exception:
    print("dim_date not found. Generating static dimension (2014-2030)...")
    static_date_table = generate_static_date_dim(spark)
    static_date_table.coalesce(1).write.mode("overwrite").parquet(f"{GOLD_BASE_PATH}dim_date")

```

if the table already exists, nothing happens.

Result: we have a file `dim_date` that contains ~6,200 rows (every day from 2014 to 2030).

Creating the symbol dimension table “dim-symbol”:

```

new_dim_symbol = df_bronze.select("symbol").distinct() \
    .withColumn("company_name", F.lit("Unknown")) \
    .withColumn("sector", F.lit("Unknown"))

```

Add two columns: `company_name` and `sector`, and fill them with the word “Unknown” (placeholder/dummy attributes)

```

try:
    existing_dim_symbol = spark.read.parquet(f"{GOLD_BASE_PATH}dim_symbol")
    final_dim_symbol = existing_dim_symbol.unionByName(new_dim_symbol) \
        .dropDuplicates(["symbol"])
    print("Merging new symbols into existing dim_symbol...")
except Exception:
    print("dim_symbol not found. Starting fresh.")
    final_dim_symbol = new_dim_symbol

```

We then check if a `dim_symbol` table already exists, and if it does, it will merge the new symbols with the already-existing ones, and remove any duplicates (this way, we can add new symbols later on).


```
final_dim_symbol.write.mode("overwrite").parquet(f"{GOLD_BASE_PATH}dim_symbol")
```

And finally, we save the dimension table to the gold layer of HDFS, overwriting anything that was present before (if any).

Creating the fact table "fact_daily_stock_prices":

```
fact_daily_stock_prices = df_bronze \
    .withColumn("date_key", F.date_format("business_date", "yyyyMMdd").cast("int")) \
    .select(
        "symbol",
        "date_key",
        "open",
        "high",
        "low",
        "close",
        "volume",
        "processing_ts"
    )
```

We select the columns that will go into the fact table (like: symbol, date_key, open, high,... etc.)

```
fact_daily_stock_prices.write.mode("overwrite").parquet(f"{GOLD_BASE_PATH}fact_daily_stock_prices")
```

And we save it to the gold layer, as well.

We then perform error handling and finish the script.

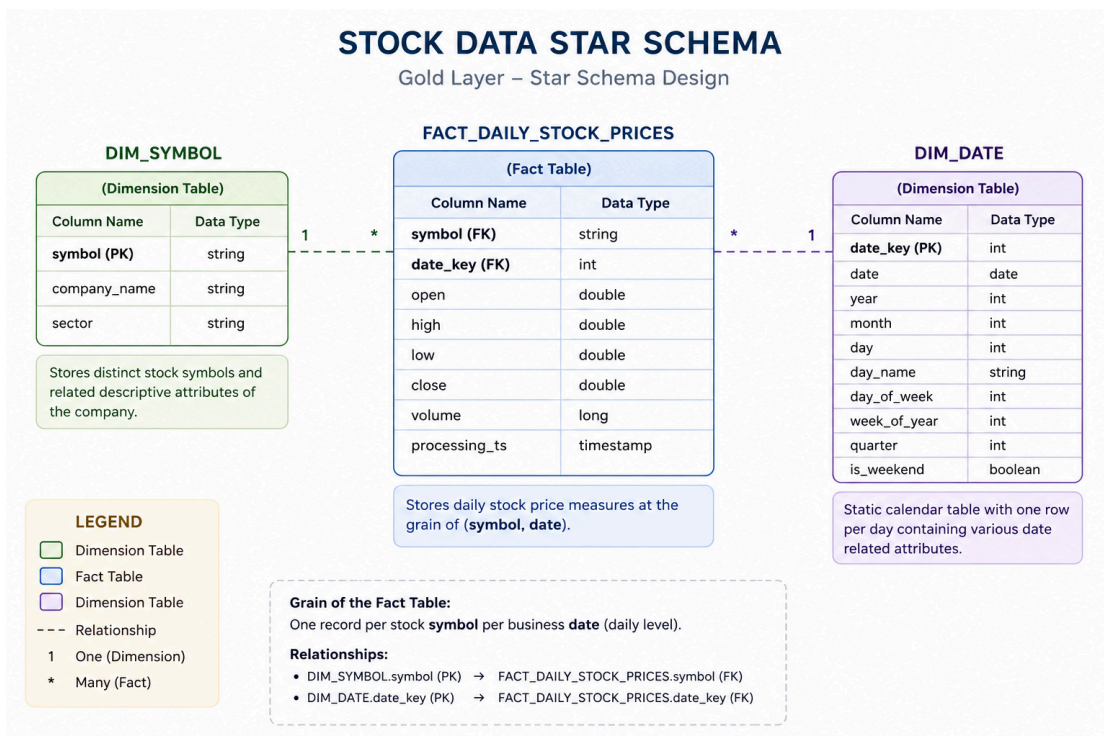
Cell output:

```

Bronze stock data loaded.
dim_date already exists. Skipping generation.
Merging new symbols into existing dim_symbol...
dim_symbol saved.
Writing fact_daily_stock_prices to Gold Layer...
Gold Layer (Star Schema) created successfully.
Spark Session Stopped.

```

Visualizing the Star Schema:



Resulting star schema after the Transformation step of ETL.

The star schema has **3** tables:

- **A calendar table (dim_date)** : a list of every single day from 2014 to 2030, with useful columns like "year", "month", "is_weekend".
- **A symbol table (dim_symbol)** : a list of all unique stock symbols, with placeholder columns ("company name", "sector").
- **A fact table (fact_daily_stock_prices)** : the actual stock prices and volumes.

▼ Loading

The goal of this step is to load the star schema (created in the transformation step) into a data warehouse (**e.g. Snowflake**)

▼ Snowflake

Snowflake is a cloud-native data warehouse. Unlike HDFS (which is just a file system), Snowflake **organises data into databases, schemas, and tables** and lets you **run SQL queries instantly**. It's where the "business user" would go to analyse the data.



▼ Resources:

<https://medium.com/@sarthak.arora.140696/what-is-snowflake-db979df7da58>

Defining Snowflake connection details:

```
sf_options = {  
    "sfURL": "DJHONTE-NJ55843.snowflakecomputing.com",  
    "sfUser": "SHAHDTAG",  
    "sfPassword": "pAvZ8KTLMUkXnMn",  
    "sfDatabase": "STOCK_DB",  
    "sfSchema": "STAR_SCHEMA",  
    "sfWarehouse": "STOCK_WH"  
}
```

This dictionary specifies the options for snowflake (e.g. the address of the snowflake account, login credentials, the created database and schema, ... etc.)

ACCOUNTADMIN • STOCK_WH (X-Small) STOCK_DB PUBLIC Share ...

```

1 CREATE OR REPLACE WAREHOUSE STOCK_WH
2 WITH WAREHOUSE_SIZE = 'XSMALL'
3 AUTO_SUSPEND = 300
4 AUTO_RESUME = TRUE;
5
6 CREATE OR REPLACE DATABASE STOCK_DB;
7
8 CREATE OR REPLACE SCHEMA STOCK_DB.STAR_SCHEMA;
9
10 SHOW DATABASES;
11 Ctrl+I to generate

```

Results (4 minutes ago)

Table Chart 5 rows 56ms

#	created_on	name	is_default	is_current	origin	owner	comment	options	rows
1	2026-05-07 09:20:11.318 -0700	SNOWFLAKE	N	N	SFCOGSOPS.SNOWHOUSE_LOCAL_GCP_ME_CENTRAL2.A				0
2	2026-05-07 09:20:13.324 -0700	SNOWFLAKE_LEARN	N	N		ACCOUNTADMIN	Created by Snowflake		1
3	2026-05-07 09:20:15.550 -0700	SNOWFLAKE_SAMP	N	N	SFSALESSHARED.SFC_SAMPLES_GCPMECENTRAL2.SAMP	ACCOUNTADMIN	Preloaded TPCH Data		0
4	2026-05-07 09:29:01.839 -0700	STOCK_DB	N	Y		ACCOUNTADMIN			1
5	2026-05-07 09:26:39.391 -0700	USER\$SHAHTAG	N	N					1

```

try:
    load_dimension_atomic("dim_date")
    load_dimension_atomic("dim_symbol")
    load_fact_append("fact_daily_stock_prices")
    print("🎉 ALL GOLD TABLES LOADED TO SNOWFLAKE!")
except Exception as e:
    print(f"❌ Loading failed: {e}")
    raise e
finally:
    spark.stop()

```

```

dim_date doesn't exist yet. Proceeding.
--- Atomic Load: STOCK_DB.STAR_SCHEMA.DIM_DATE ---
SUCCESS: STOCK_DB.STAR_SCHEMA.DIM_DATE loaded atomically.
--- Atomic Load: STOCK_DB.STAR_SCHEMA.DIM_SYMBOL ---
SUCCESS: STOCK_DB.STAR_SCHEMA.DIM_SYMBOL loaded atomically.
--- Append Load: STOCK_DB.STAR_SCHEMA.FACT_DAILY_STOCK_PRICES ---
SUCCESS: STOCK_DB.STAR_SCHEMA.FACT_DAILY_STOCK_PRICES appended.
🎉 ALL GOLD TABLES LOADED TO SNOWFLAKE!

```

Cell output indicating that we have loaded the tables of the star scheme to Snowflake

Preview: STOCK_DB.STAR_SCHEMA.FACT_DAILY... +

Table Chart 100 rows 972ms

	SYMBOL	DATE_KEY	OPEN	HIGH	LOW	CLOSE	VOLUME	PROCESSING_TS
	AAL	20171215	3.655	3.69	3.5	3.51	581512	5/6/2026
1	AAL	20171215	50.67	51.5	50.4	51.06	9074165	2026-05-06 13:07:16.000
2	AAPL	20171215	173.63	174.17	172.46	173.97	40169307	2026-05-06 13:07:16.000
3	AAP	20171215	101.18	101.72	100.17	100.7	1617013	2026-05-06 13:07:16.000
4	ABV	20171215	97.06	97.88	96.64	97.45	11564691	2026-05-06 13:07:16.000
5	ABC	20171215	91.17	94.06	90.78	93.17	4153406	2026-05-06 13:07:16.000
6	ABT	20171215	55.01	55.56	54.84	55.5	14655482	2026-05-06 13:07:16.000
7	ACN	20171215	151.56	152.9	150.66	152.66	3498820	2026-05-06 13:07:16.000
8	ADBE	20171215	179.36	179.98	174.3004	177.51	9191676	2026-05-06 13:07:16.000
9	ADI	20171215	86.07	87.205	85.29	86.69	3765981	2026-05-06 13:07:16.000
10	ADM	20171215	39.85	40.96	39.775	40.56	9689797	2026-05-06 13:07:16.000
11	ADP	20171215	117.75	118.69	116.34	118.19	3286913	2026-05-06 13:07:16.000
12	ADSK	20171215	106.55	108.93	106.47	108.4	2704189	2026-05-06 13:07:16.000
13	ADS	20171215	237	240.88	230.53	239.81	1047149	2026-05-06 13:07:16.000
14	AEE	20171215	60.98	61.44	60.83	60.94	2832234	2026-05-06 13:07:16.000
15	AEP	20171215	76.33	76.9	75.81	76.54	3894801	2026-05-06 13:07:16.000
16	AES	20171215	10.7	11.02	10.68	10.8	22897013	2026-05-06 13:07:16.000
17	AET	20171215	178.92	180.62	178.16	179.7	5656750	2026-05-06 13:07:16.000
18	AET	20171215	89.56	89.67	88.39	89.76	3016576	2026-05-06 13:07:16.000

Feedback

Preview of the fact table in snowflake

Database Explorer

HORIZON CATALOG

Databases Data products

Search



Filter

- > SNOWFLAKE
- > SNOWFLAKE_LEARNING_DB
- > SNOWFLAKE_SAMPLE_DATA
- ✓ STOCK_DB
 - > INFORMATION_SCHEMA
 - > PUBLIC
 - > STAR_SCHEMA
- > USER\$SHAHDTAG

A screenshot from Snowflake's Data Explorer, showing STOCK_DB

Database Explorer

HORIZON CATALOG

DatabasesData products

Search

Filter

SNOWFLAKE

SNOWFLAKE_LEARNING_DB

SNOWFLAKE_SAMPLE_DATA

STOCK_DB

INFORMATION_SCHEMA

PUBLIC

STAR_SCHEMA

Tables

DIM_DATE

DIM_SYMBOL

FACT_DAILY_STOCK_PRICES

USERS\$SHAHDTAG

STOCK_DB / STAR_SCHEMA

SchemaACCOUNTADMIN1 hour ago

Schema DetailsTables

3 Tables

Search

All Tables

NAME	TYPE	OWNER	ROWS	BYTES	CREATED	
DIM_DATE	Table	ACCOUNTADMIN	6.2K	48.5KB	15 minutes ago	...
DIM_SYMBOL	Table	ACCOUNTADMIN	505	2.5KB	15 minutes ago	...
FACT_DAILY_STOCK...	Table	ACCOUNTADMIN	1.5M	23.2MB	49 minutes ago	...

Preview of all three tables from Snowflake's Database Explorer

Database Explorer

HORIZON CATALOG

DatabasesData products

Search

Filter

SNOWFLAKE

SNOWFLAKE_LEARNING_DB

SNOWFLAKE_SAMPLE_DATA

STOCK_DB

INFORMATION_SCHEMA

PUBLIC

STAR_SCHEMA

Tables

DIM_DATE

DIM_SYMBOL

FACT_DAILY_STOCK_PRICES

USERS\$SHAHDTAG

STOCK_DB / STAR_SCHEMA / DIM_DATE

TableACCOUNTADMIN16 minutes ago6.2K48.5KB

Table DetailsColumnsData PreviewCopy History

COMPUTE_WH

100 of 6.2K Rows • Updated just now

	DATE_KEY	DATE	YEAR	MONTH	DAY	DAY_NAME	DAY_OF_WEEK	WEEK_OF_YEAR	QUARTER	IS_WEEKEND
1	20140101	2014-01-01	2014	1	1	Wednesday	4	1	1	FALSE
2	20140102	2014-01-02	2014	1	2	Thursday	5	1	1	FALSE
3	20140103	2014-01-03	2014	1	3	Friday	6	1	1	FALSE
4	20140104	2014-01-04	2014	1	4	Saturday	7	1	1	TRUE
5	20140105	2014-01-05	2014	1	5	Sunday	1	1	1	TRUE
6	20140106	2014-01-06	2014	1	6	Monday	2	2	1	FALSE
7	20140107	2014-01-07	2014	1	7	Tuesday	3	2	1	FALSE
8	20140108	2014-01-08	2014	1	8	Wednesday	4	2	1	FALSE
9	20140109	2014-01-09	2014	1	9	Thursday	5	2	1	FALSE
10	20140110	2014-01-10	2014	1	10	Friday	6	2	1	FALSE
11	20140111	2014-01-11	2014	1	11	Saturday	7	2	1	TRUE
12	20140112	2014-01-12	2014	1	12	Sunday	1	2	1	TRUE
13	20140113	2014-01-13	2014	1	13	Monday	2	3	1	FALSE

Preview of the DIM_DATE table from Snowflake's Database Explorer

Database Explorer

HORIZON CATALOG

DatabasesData products

Search

Filter

SNOWFLAKE

SNOWFLAKE_LEARNING_DB

SNOWFLAKE_SAMPLE_DATA

STOCK_DB

INFORMATION_SCHEMA

PUBLIC

STAR_SCHEMA

Tables

DIM_DATE

DIM_SYMBOL

FACT_DAILY_STOCK_PRICES

USERS\$SHAHDTAG

STOCK_DB / STAR_SCHEMA / DIM_SYMBOL

TableACCOUNTADMIN23 minutes ago5052.5KB

Table DetailsColumnsData PreviewCopy History

COMPUTE_WH

100 of 505 Rows • Updated just now

	SYMBOL	COMPANY_NAME	SECTOR
1	A	Unknown	Unknown
2	AAL	Unknown	Unknown
3	AAP	Unknown	Unknown
4	AAPL	Unknown	Unknown
5	ABV	Unknown	Unknown
6	ABC	Unknown	Unknown
7	ABT	Unknown	Unknown
8	ACN	Unknown	Unknown
9	ADBE	Unknown	Unknown
10	ADI	Unknown	Unknown
11	ADM	Unknown	Unknown
12	ADP	Unknown	Unknown
13	ADS	Unknown	Unknown
14	ADSK	Unknown	Unknown

Preview of the DIM_SYMBOL table from Snowflake's Database Explorer

▼ Orchestration (Airflow)

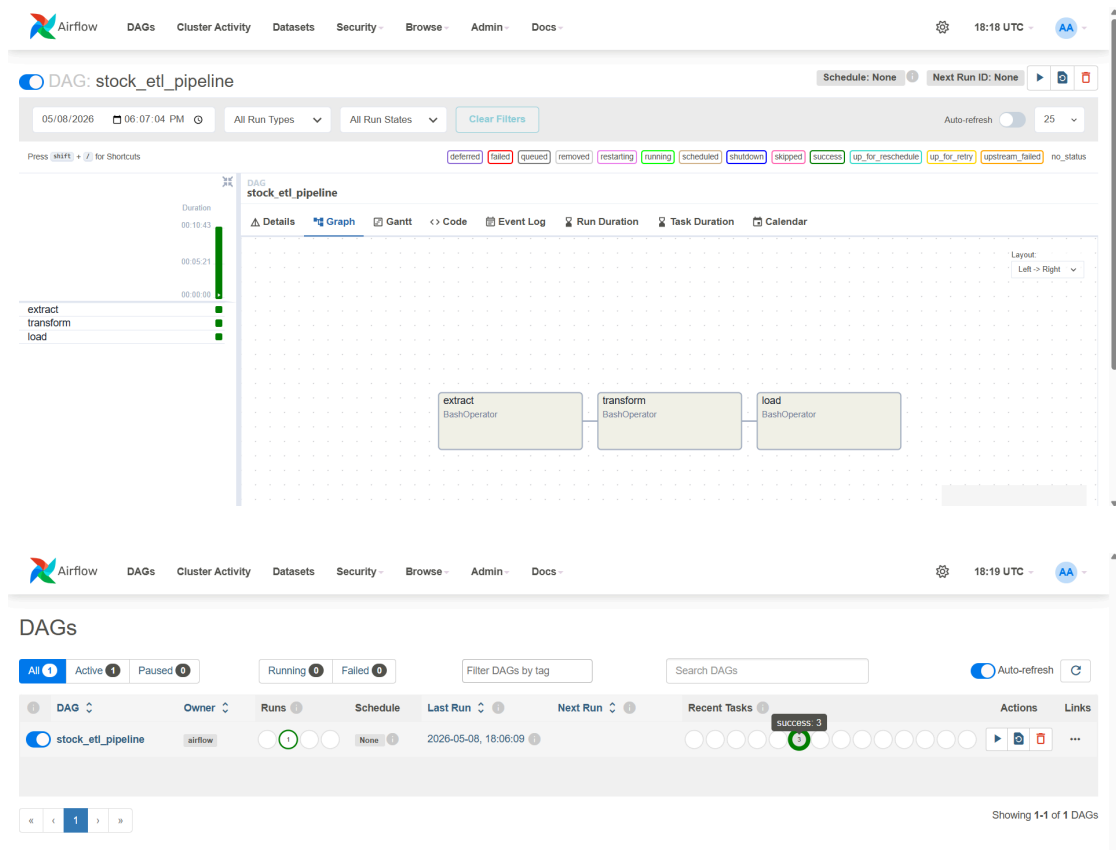
▼ Airflow



Airflow is a platform used for scheduling batch workflows, through the definition of a DAG (**D**irected **A**cyclic **G**raphs).

A DAG represents **the collection of tasks that need to be run to process a raw batch of data.**

▼ Successful DAG screenshots:



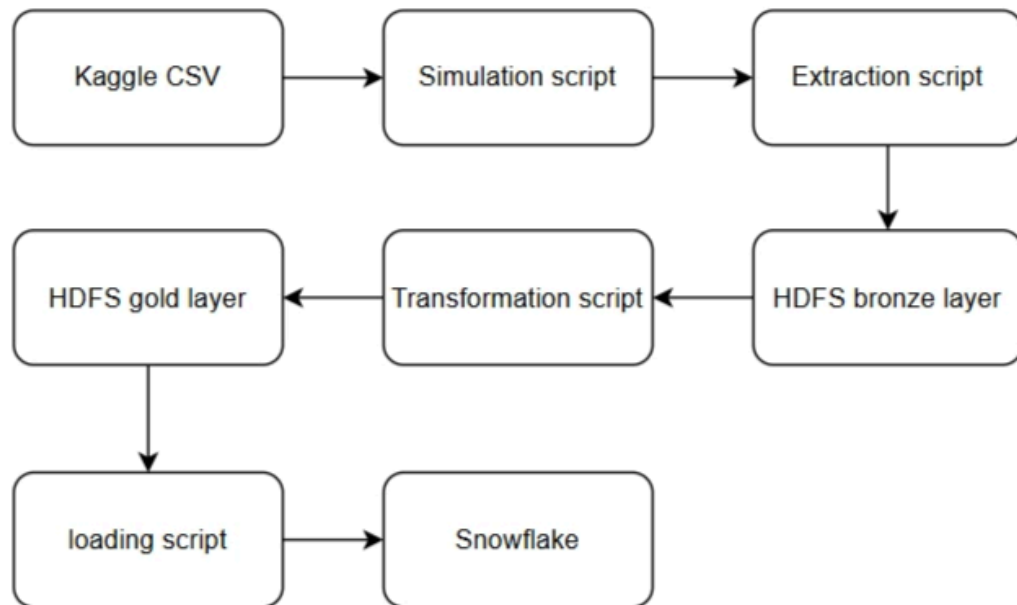
The first screenshot shows the Airflow web interface for a DAG named 'stock_etl_pipeline'. The top navigation bar includes links for DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. The main header displays the DAG name, a 'Schedule: None' button, and a 'Next Run ID: None' button. Below this, there are filters for '05/08/2026' and '06:07:04 PM', and a 'Clear Filters' button. A status bar shows various task states: deferred, failed, queued, removed, restarting, running, scheduled, shutdown, skipped, success, up_for_reschedule, up_for_retry, upstream_failed, and no_status. The main content area shows a graph of the DAG with three tasks: 'extract' (BashOperator), 'transform' (BashOperator), and 'load' (BashOperator). The 'extract' task is highlighted, showing its duration as 00:05:21. The second screenshot shows the 'DAGs' list in the Airflow web interface. It includes a table with columns for DAG, Owner, Runs, Schedule, Last Run, Next Run, Recent Tasks, Actions, and Links. The 'stock_etl_pipeline' DAG is listed with a status of 'success: 3' and a last run time of '2026-05-08, 18:06:09'.

The screenshot shows the Airflow web interface for the 'stock_etl_pipeline' DAG. The top navigation bar includes links for DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. The right side shows the current time as 18:20 UTC. Below the navigation bar, there's a status bar with various task states like deferred, failed, queued, removed, restarting, running, scheduled, shutdown, skipped, success, up_for_reschedule, up_for_retry, upstream_failed, and no_status. The main content area is divided into two tabs: 'Details' and 'Code'. The 'Code' tab is active, displaying the DAG definition in Python. The code defines a DAG with a dag_id of 'stock_etl_pipeline', a start_date of '2024-06-01', a schedule interval of 'None', and a catchup of 'False'. It includes two tasks: 'extract' and 'transform', both using the 'BashOperator' class. The 'extract' task runs a 'docker exec spark-jupyter spark-submit' command, and the 'transform' task runs a 'python' command. The 'load' task is also present but its command is not visible in the screenshot.

The screenshot shows the Airflow web interface for the 'stock_etl_pipeline' DAG, specifically the 'DAG Runs Summary' table. The top navigation bar is the same as the previous screenshot. The right side shows the current time as 18:21 UTC. Below the navigation bar, there's a status bar with various task states. The main content area is divided into two tabs: 'Details' and 'Code'. The 'Details' tab is active, displaying the DAG Runs Summary table. The table has two columns: 'Metric' and 'Value'. The metrics and their values are: Total Runs Displayed (1), Total success (1), First Run Start (2026-05-08, 18:06:10 UTC), Last Run Start (2026-05-08, 18:06:10 UTC), Max Run Duration (00:10:43), Mean Run Duration (00:10:43), and Min Run Duration (00:10:43). The 'DAG Summary' section is also visible at the bottom of the table.

▼ Architecture Diagram

For this project:



If It was a real system:

